

PROMETEO
by thePower

TRABAJO DE DOCUMENTACIÓN
Sistemas Informáticos
1º DAM



Proyecto Intermodular

Asignatura: Sistemas Informáticos

Fecha. límite entrega: 15/05/2026

Profesor: David Torres

Miembros: Alfonso Bercedo Núñez

Víctor Tierno Corbatón

David Domingo Guevara

Índice

1. Introducción
2. Entorno de ejecución de aplicaciones Java
 - 2.1 ¿Qué necesita un equipo para ejecutar Java?
 - 2.2 Diferencia entre JDK y JRE
 - 2.3 Instalación de Java en Windows y Linux
 - 2.4 Variables JAVA_HOME y PATH
3. Creación de instaladores
 - 3.1 ¿Qué es un instalador?
 - 3.2 Herramientas para crear instaladores Java
 - 3.3 Elementos incluidos en un instalador
4. Servicios, APIs y comunicación en red
 - 4.1 ¿Qué es una API?
 - 4.2 Spring Boot
 - 4.3 Conceptos básicos de red
5. Contenedores con Docker
 - 5.1 ¿Qué es Docker?
 - 5.2 Diferencia entre máquina virtual y contenedor
 - 5.3 Conceptos básicos de Docker
 - 5.4 Relación entre Docker y Java
6. Aplicaciones frontend con Angular y React
 - 6.1 ¿Qué es un frontend?
 - 6.2 Angular y React
 - 6.3 Node.js, npm y servidor de desarrollo
 - 6.4 Build y despliegue
 - 6.5 Relación con backend Java
7. Conclusión
8. Bibliografía

1. Introducción

En el desarrollo de software actual, crear una aplicación no solo consiste en programar, sino también en conseguir que se pueda ejecutar y funcionar bien en distintos dispositivos, sistemas operativos y redes. Actualmente, las aplicaciones usan distintas tecnologías que permiten desarrollar, conectar y desplegar las aplicaciones de forma eficiente.

El objetivo de este trabajo es analizar cómo una aplicación Java pasa desde su creación hasta su funcionamiento en un entorno real. Vamos a tratar temas como la instalación y configuración de Java, la creación de instaladores, el uso de APIs, Docker y la integración con tecnologías frontend como Angular y React. Además, vamos a explicar conceptos importantes, entre ellos la máquina virtual de Java, las variables de entorno, los puertos, los protocolos de red y los servidores web.

2. Entorno de ejecución de aplicaciones

Java 2.1 Ejecutar Java en un dispositivo

Como ya hemos visto, para ejecutar aplicaciones Java es necesario disponer de un entorno compatible con Java. El componente más importante es la JVM (Java Virtual Machine), que es una máquina virtual encargada de interpretar y ejecutar el código Java compilado. Cuando un programador desarrolla una aplicación Java, el código fuente se compila en un formato llamado bytecode. Este bytecode no depende del sistema operativo, sino que es ejecutado por la JVM. Gracias a esto, Java puede funcionar en Windows, Linux o macOS sin necesidad de modificar el programa. La JVM permite que Java sea un lenguaje multiplataforma. Por ello se suele utilizar la frase: "Write once, run anywhere".

2.2 Diferencia entre JDK y JRE

Dentro de Java existen varios componentes necesarios para poder crear y ejecutar aplicaciones. Los más importantes son el JDK y el JRE, aunque cada uno tiene una función diferente.

El JDK es el paquete que se utiliza para desarrollar programas en Java. Incluye las herramientas necesarias para escribir, compilar y probar aplicaciones. Entre sus elementos principales están el compilador `javac`, librerías de Java, herramientas de desarrollo, el JRE integrado. Por otro lado, el JRE sirve únicamente para ejecutar aplicaciones Java ya creadas. Contiene la máquina virtual de Java (JVM) y las librerías básicas necesarias para que los programas funcionen correctamente. A diferencia del JDK, el JRE no incluye herramientas para programar ni compilar código.

2.3 Instalación de Java en Windows y Linux

Para poder desarrollar y ejecutar aplicaciones Java es necesario instalar previamente Java en el sistema operativo. El proceso cambia ligeramente dependiendo de si se utiliza Windows o Linux.

En Windows, la instalación suele hacerse mediante un asistente gráfico.

Los pasos principales son:

1. Descargar el JDK desde la página oficial de Oracle o desde OpenJDK.
2. Ejecutar el archivo instalador.
3. Seguir los pasos indicados por el asistente de instalación.
4. Configurar las variables de entorno si fuese necesario.

Una vez terminado el proceso, Java quedará instalado y listo para utilizarse.

En distribuciones basadas en Ubuntu, Java puede instalarse fácilmente desde la terminal utilizando los repositorios del sistema. Los comandos más utilizados son “`sudo apt update`” y “`sudo apt install openjdk-17-jdk`”.

Después de instalar Java, es recomendable comprobar que funciona correctamente. Para ello, desde la terminal o el símbolo del sistema se puede ejecutar “java -version” y debería de parecer la versión de java instalada, dando por supuesto que está instalado.

2.4 Variables JAVA_HOME y PATH

Para que Java funcione correctamente en el sistema, es importante configurar algunas variables de entorno. Las más utilizadas son JAVA_HOME y PATH.

La variable JAVA_HOME indica la ubicación donde está instalado el JDK de Java dentro del ordenador. Muchas aplicaciones y herramientas de desarrollo utilizan esta variable para localizar Java correctamente.

Variable PATH La variable PATH permite ejecutar comandos de Java desde cualquier carpeta de la terminal o del símbolo del sistema, sin necesidad de escribir la ruta completa.

Gracias a esto, comandos como java o javac pueden utilizarse directamente desde la consola y si JAVA_HOME o PATH no están configuradas correctamente, pueden aparecer diferentes errores.

3. Creación de instaladores

3.1 ¿Qué es un instalador?

Un instalador es un programa que permite instalar una aplicación de forma automática en un ordenador. Su función es facilitar el proceso al usuario para que no tenga que copiar archivos ni realizar configuraciones manualmente.

En aplicaciones Java, en lugar de entregar únicamente un archivo .jar, muchas veces se utiliza un instalador para que la aplicación quede preparada y configurada correctamente desde el principio.

Entre las principales ventajas de utilizar instaladores se encuentran:

- Realizar una instalación guiada paso a paso.
- Crear accesos directos automáticamente.
- Configurar el sistema de forma automática.
- Incluir librerías y dependencias necesarias.
- Añadir un desinstalador para eliminar la aplicación fácilmente.

3.2 Herramientas para crear instaladores Java

Existen diferentes herramientas que permiten crear instaladores y ejecutables para aplicaciones Java. Cada una ofrece distintas opciones según las necesidades del proyecto.

Inno Setup es una herramienta gratuita muy utilizada en Windows para crear instaladores. Permite generar asistentes de instalación sencillos y personalizables, además de crear accesos directos y desinstaladores.

install4j es una herramienta profesional orientada a aplicaciones Java. Ofrece más opciones de configuración y permite crear instaladores para varios sistemas operativos.

jpackage es una herramienta oficial incluida en las versiones recientes de Java. Sirve para generar instaladores y paquetes ejecutables de forma más integrada con el propio JDK.

Launch4j se utiliza para convertir aplicaciones .jar en archivos ejecutables .exe para Windows. De esta forma, los programas pueden iniciarse como cualquier otra aplicación del sistema operativo.

3.3 Elementos incluidos en un instalador

Un instalador no solo copia la aplicación en el ordenador, sino que también puede incluir diferentes elementos necesarios para que el programa funcione correctamente desde el primer momento.

Entre los componentes más habituales se encuentran:

- Los archivos principales de la aplicación.
- Dependencias necesarias para su funcionamiento.

- Librerías adicionales.
- Configuraciones iniciales del programa.
- Creación de accesos directos.
- Selección de la ruta de instalación.
- Un desinstalador para eliminar la aplicación fácilmente.

Gracias a todo esto, los usuarios pueden instalar la aplicación de manera más rápida y sencilla, evitando errores o configuraciones manuales innecesarias.

4. Servicios, APIs y comunicación en red

4.1 ¿Qué es una API?

Una API (Application Programming Interface) es un sistema que permite la comunicación entre diferentes aplicaciones o programas. Gracias a las APIs, distintos servicios pueden intercambiar información de forma organizada y automática.

Normalmente, las APIs funcionan siguiendo el modelo cliente-servidor (cliente realiza una petición, servidor procesa esa solicitud, servidor devuelve una respuesta con los datos correspondientes).

Este sistema se utiliza constantemente en aplicaciones actuales. Por ejemplo, una aplicación móvil puede conectarse a un servidor desarrollado en Java para solicitar información mediante una API. De esta forma, ambos programas pueden comunicarse aunque estén ejecutándose en dispositivos diferentes.

4.2 Spring Boot

Spring Boot es un framework de Java que se utiliza principalmente para crear aplicaciones backend y APIs REST de una forma más sencilla y rápida.

Su objetivo es reducir la cantidad de configuración necesaria para poder empezar a desarrollar, lo que hace que el proceso sea más ágil.

Ahora mismo, Spring Boot es una de las tecnologías más utilizadas dentro del desarrollo backend con Java, ya que permite crear aplicaciones robustas de manera bastante eficiente.

4.3 Conceptos básicos de red

Para entender cómo se comunican las aplicaciones en red, es importante conocer algunos conceptos básicos.

La dirección IP es un identificador que tiene cada dispositivo dentro de una red. Sirve para poder localizarlo y enviarle información correctamente.

Un puerto es un “canal” de comunicación que utilizan las aplicaciones para intercambiar datos dentro de un mismo dispositivo o servidor.

HTTP y HTTPS son protocolos que se utilizan para la comunicación en la web. HTTP permite la transferencia de información. HTTPS es la versión segura, ya que añade cifrado para proteger los datos.

Peticiones GET y POST En las APIs web se utilizan diferentes tipos de peticiones:

GET → se utiliza para solicitar información.

POST → se utiliza para enviar información al servidor.

Las respuestas de las APIs suelen enviarse en formato JSON, ya que es un formato ligero y fácil de leer tanto para las personas como para las aplicaciones.

5. Contenedores con Docker

5.1 ¿Qué es Docker?

Docker es una herramienta que permite ejecutar aplicaciones dentro de contenedores.

Un contenedor incluye todo lo necesario para que una aplicación funcione (aplicación propia, librerías, dependencias, configuración).

Gracias a esto, la aplicación puede ejecutarse igual en cualquier equipo, sin problemas de compatibilidad.

5.2 Diferencia entre máquina virtual y contenedor

Una máquina virtual incluye un sistema operativo completo, consume más recursos y es más pesada. Por otro lado, un contenedor comparte el sistema operativo del equipo host, es más ligero y rápido y se inicia casi al instante.

5.3 Conceptos básicos de Docker

Imagen: plantilla a partir de la cual se crean contenedores.

Contenedor: instancia en ejecución de una imagen.

Dockerfile: archivo donde se indican los pasos para crear una imagen.

Volumen: permite guardar datos de forma persistente.

Puerto expuesto: permite acceder a la aplicación desde fuera del contenedor.

5.4 Relación entre Docker y Java

Una aplicación Java, por ejemplo con Spring Boot, se puede empaquetar dentro de una imagen Docker y ejecutarse en cualquier equipo que tenga Docker instalado.

Esto evita tener que configurar Java o dependencias manualmente en cada entorno.

Sus principales ventajas son:

- Facilita el despliegue
- Mejora la portabilidad
- Permite escalar la aplicación más fácilmente

6. Aplicaciones frontend con Angular y React

6.1 ¿Qué es un frontend?

El frontend es la parte de la aplicación que ve y utiliza el usuario. Normalmente se ejecuta en el navegador.

6.2 Angular y React

- Angular: framework desarrollado por Google.
- React: biblioteca desarrollada por Meta.

Ambos se utilizan para crear interfaces web modernas y dinámicas.

6.3 Node.js, npm y servidor de desarrollo

- Node.js: permite ejecutar JavaScript fuera del navegador.
- npm: gestor de paquetes para instalar dependencias.
- Servidor de desarrollo: se usa durante la programación para probar la aplicación en local.

6.4 Build y despliegue

Cuando la aplicación está terminada, se genera una versión optimizada para producción con “npm run build”.

Esto crea archivos finales de HTML, CSS, JavaScript.

Después se pueden subir a servidores como Apache, Nginx, Vercel y Netlify

6.5 Relación con backend Java

El frontend y el backend se comunican mediante APIs REST.

Os voy a poner un ejemplo: El frontend solicita datos, el backend en Spring Boot procesa la petición y el servidor devuelve una respuesta en

formato JSON. De esta forma, la parte visual y la lógica del servidor están separadas.

7. Conclusión

Las aplicaciones actuales combinan varias tecnologías que trabajan juntas. Java sigue siendo muy importante gracias a la JVM y a su capacidad para funcionar en distintos sistemas.

Herramientas como Docker facilitan el despliegue de aplicaciones en diferentes entornos, y frameworks como Angular o React permiten crear interfaces modernas para el usuario.

Entender todo el proceso, desde el desarrollo hasta el despliegue, es clave en ciclos como DAM o DAW, ya que ayuda a comprender cómo funcionan realmente las aplicaciones en un entorno profesional.

8. Bibliografía

- [Java Documentation - Get Started](#)
- [Spring Boot](#)
- [The Essential Spring IDE – IntelliJ IDEA](#)
- [Docker Docs](#)
- [Home • Angular](#)
- [React](#)
- [Node.js — Run JavaScript Everywhere](#)
- [HTTP: Hypertext Transfer Protocol | MDN](#)